

UNIVERSITÉ PARIS CITÉ

PROJET INFORMATIQUE (JAVA)

Rapport de Projet

Implémentation en JAVA d'un jeu Football Mana-J-er

CB2 - C

ARNAUD Hugo
FOALEM Ruben
HARISSE Sofyane

Année 2025 - 2026

Table des matières

1	Gestion du dépôt	3
1.1	Branches	3
1.2	Labels	3
2	Jalons	4
2.1	Jalon 0 : Initialisation du projet GitLab	4
2.2	Jalon 1 : Ajout des classes de base + BDD	4
2.3	Jalon 2 : Création de l'interface graphique	4
2.4	Jalon 3 : Amélioration BDD, modèle et vue graphique	4
2.5	Jalon 4-5 : Mise en place du système monétaire et du contrôleur	5
2.6	Jalon 6 : Amélioration du projet, mise en place d'extensions	5
3	Choix techniques	5
3.1	Bibliothèques	5
3.2	Structure du code	5
3.3	Patrons de conception	6
3.4	Système de sauvegarde	6
3.5	Gestionnaire de langue	6
4	Problèmes	6
5	Annexes	7
5.1	Structure des classes	7
5.2	Architecture de la base de données	8
5.3	Outils	9
5.4	Dépendances	9
5.5	Images	9
5.6	Sons	9

1 Gestion du dépôt

1.1 Branches

Comme décrit dans le fichier CONTRIBUTING, le projet est divisé en plusieurs branches :

- **dev** : Branche par défaut qui contient toutes les nouvelles fonctionnalités, tests et documentation, sans bugs majeurs.
- **database** : Branche contenant des modifications en lien avec la base de données.
- **documentation** : Branche contenant les ajouts de documentation (README, CONTRIBUTING, Javadoc, schémas, etc.).
- **feature** : Branche avec des fonctionnalités testées.
- **tests** : Branche contenant les ajouts de tests.

Ces branches servent de point de référence et ont donc été marquées comme protégées (pas de suppression programmées).

Nous avons aussi utilisés des branches intermédiaires, supprimées à la fin de chaque jalon. Elles ont eu pour but de spécifier le travail effectué, et dans quel package (app, controller, database, model, test ou view).

1.2 Labels

Afin de bien séparer les différentes tâches en fonction de leur catégorie, nous avons créés des labels. Ceux-ci nous permettent également de mieux retrouver un label particulier. Ils ont été divisés en 2 catégories, prioritaires et autres :

Prioritaires :

- URGENT (A traiter en priorité)
- BUGFIX (Bug(s) à résoudre)
- PRIORITAIRE (MR ou ticket le plus urgent)

Autres :

- BDD (Ticket ou MR concernant la base de données)
- Compte Rendu (Compte rendu de chaque jalon)
- Documentation (Ajout ou modification de fichier(s) destiné(s) à la documentation)
- En cours (Ticket attribué et démarré)
- En revue (Attente d'approbation)
- Fonctionnalité (Ajout ou modification de fichier(s) pour la fonctionnalité du jeu)
- Organisation (Distribution de tâche(s))
- Planning (Planification de tâche(s))
- Proposition (Ticket ou MR en attente de validation de démarrage)
- Structure (Ajout ou suppression de dossier(s))
- Test (Ajout ou modification de fichier(s) de tests)

2 Jalons

— 2.1 Jalon 0 : Initialisation du projet GitLab

Objectifs :

Mettre en place l'environnement de travail et organiser le projet.

Mise en place de l'environnement de travail collectif et organisation du projet.

Choix du Workflow.

Programmation des jalons à venir.

— 2.2 Jalon 1 : Ajout des classes de base + BDD

Objectifs :

Ajout des classes principales du projet (Player, Team, Manager...).

Choix de la Base de Données.

Ajout des classes pour interagir avec la base de données (DAO).

Ajout des premiers tests de classes.

Création du modèle.

Création de la structure MVC (Modèle-Vue-Contrôleur) : Main, DatabaseManager, GameModel, GameView, GameController.

— 2.3 Jalon 2 : Création de l'interface graphique

Objectifs :

Correction des problèmes en lien avec la base de données.

Réflexion sur les interactions entre les différents panel.

Création de l'interface graphique.

— 2.4 Jalon 3 : Amélioration BDD, modèle et vue graphique

Objectifs :

Amélioration de notre base de données.

Amélioration de l'interface graphique.

Enrichissement du modèle avec l'ajout de nouvelles classes.

— 2.5 Jalon 4-5 : Mise en place du système monétaire et du contrôleur

Objectifs :

Implémentation du système de budget (achat/vente de joueurs)

Mise en place de la gestion d'équipe. (Modèle)

Ajout d'attributs dans la base de données (Database + Modèle)

Algorithme de simulation des matchs basé sur les statistiques (passe, vitesse, tir, défense)

Ajout des infrastructures (controller + model + view -> amélioration du jeu)

— 2.6 Jalon 6 : Amélioration du projet, mise en place d'extensions

Objectifs d'amélioration :

Amélioration de l'algorithme de match.

Révision du système de contrat de joueurs.

Ajout d'un calendrier simulant une saison.

Révision de la méthode d'amélioration des joueurs + infrastructures.

Amélioration du système de sauvegarde local.

Ajout du Championnat.

Ajout des sons/images.

Mise en place du système de courrier (mails)

Mise en place d'un système de quêtes pour aider dans la progression (récompenses financières)

3 Choix techniques

3.1 Bibliothèques

Pour concevoir une interface graphique interactive, nous avons utilisé Java Swing et AWT pour la gestion des dispositions (layouts). La persistance des données est assurée par une base de données PostgreSQL hébergée sur Supabase, accessible via le pilote JDBC. Nous avons également intégré Gson (Google) pour la sérialisation de données locales (configurations, mails) dans des fichiers json facilement lisibles et Dotenv pour sécuriser les identifiants de connexion.

3.2 Structure du code

Afin d'avoir un code propre et facilement modifiable, nous avons choisi d'adopter le pattern MVC. Cette dissociation sépare la logique métier (GameModel, Team, Player, Manager) de l'affichage graphique (GameView et ses différents panels). La coordination est assurée par un GameController centralisé qui délègue les tâches spécifiques à des

sous-contrôleurs (MatchController, InfrastructureController, SoundController), garantissant un code modulaire et évolutif. L'application utilise également des classes utilitaires statiques (HRSColors, HRSFonts) présentent dans le package view/visuals pour uniformiser le thème graphique.

3.3 Patrons de conception

L'idée générale de la structure des classes et de la base de données à été pensé grâce aux méthodes UML (schématiser les classes et leurs attributs/méthodes et les relier entre elles) et JMerise (création de tables et indication des liaisons pour les clés). En complément de la MVC, nous avons implémenté le pattern DAO (Data Access Object) (ex : DataPlayer, DataTeam) dans le package database/dao/ pour isoler les requêtes SQL de la logique métier.

3.4 Système de sauvegarde

Afin d'avoir une persistance des données, les données importantes sont sauvegardées dans la base de données Supabase, accessible par chacun en ligne) tandis que les autres sont stockées des fichiers json (configuration de l'utilisateur, calendrier, mails et championnat). Les paramètres de connexion dépendent d'un fichier d'environnement (.env) pour nous et d'un fichier de propriétés présent dans le code pour les personnes externes.

3.5 Gestionnaire de langue

La traduction est gérée de manière centralisée par la classe view/visuals/HRSLanguages. Elle s'appuie sur des fichiers de propriétés (src/main/resources/languages/) qui ne contiennent que du texte. Ce module supporte actuellement quatre langues (anglais, espagnol, français, italien) et rend l'ajout d'une nouvelle langue très simple (création d'un nouveau fichier et ajout du choix dans le SettingsPanel). Le changement de langue déclenche une mise à jour via une méthode présente dans la vue principale (GameView) sur tous les composants actifs, permettant une bascule instantanée sans redémarrage de l'application.

4 Problèmes

- Limite de stockage : Initialement hébergée sur Neon, la base de données a rapidement atteint ses quotas de stockage gratuits en raison du volume de données générée par la création des joueurs/équipes.
Solution : Migration de base, NEON -> SUPABASE, plus de stockage et une interface plus complète (editeur SQL et visualisation de tables, entre autres)
- Accès à la base de données depuis l'exécution du JAR.
Solution : Ajout des dépendances nécessaires dans le fichier pom.xml (configuration de Maven)
- Exécution des tests unitaires (envoi de requêtes à la base)
Solution : Aucune solution trouvée malgré de multiples tentatives
Compensation : Une simple vérification visuelle sur le site de l'hébergeur suffit

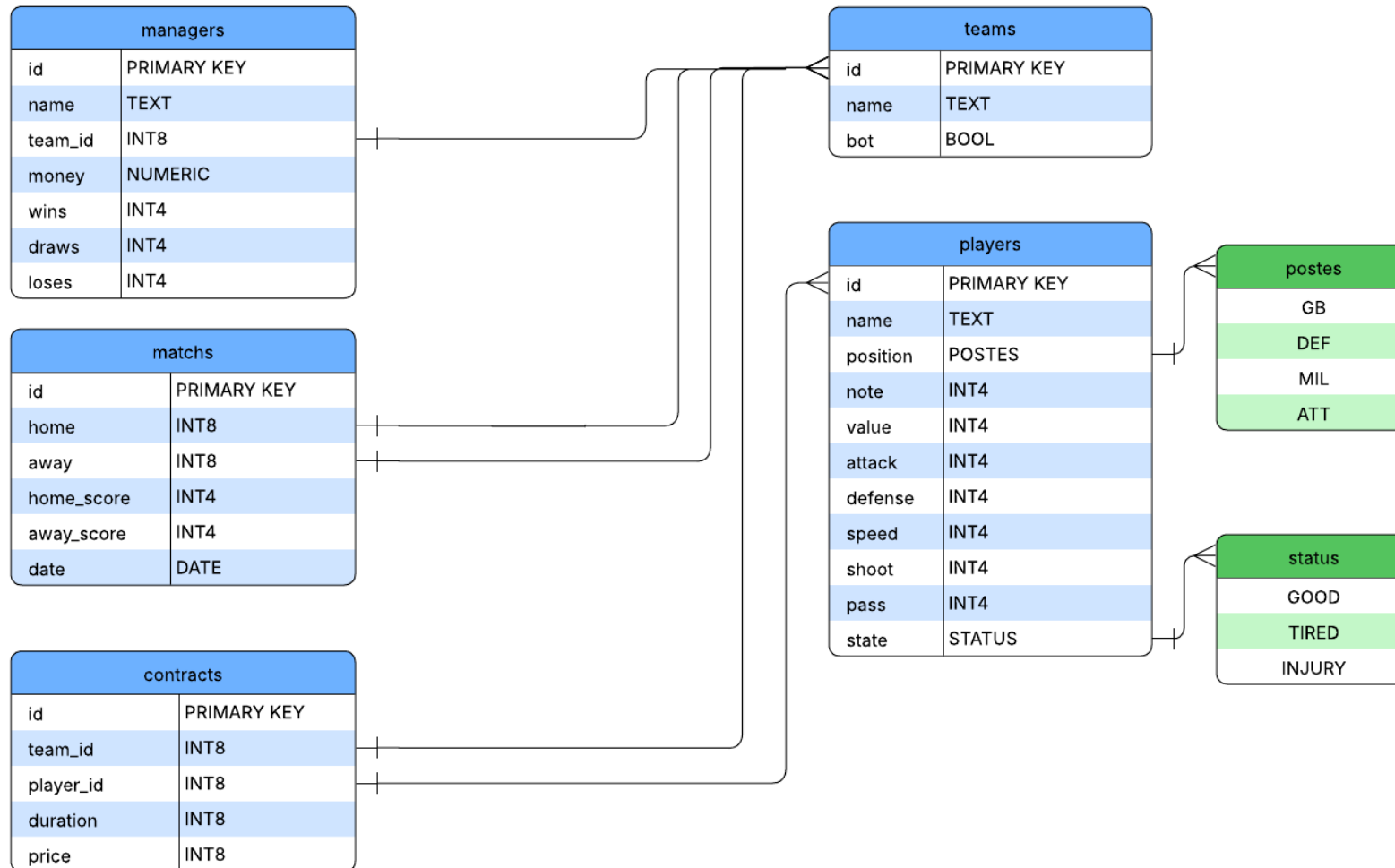
5 Annexes

5.1 Structure des classes

src/main/java/com/kahrs				
app	controller	database	model	view
CalendarManager ChampionshipManager MailManager Main QuestsManager UserConfig	CalendarController GameController InfrastructureController MatchController SoundController	dao/ DataContract DataManager DataMatch DataPlayer DataTeam DatabaseManager	CalendarEvent Championship Contract GameModel Infrastructure Mail Manager Match Player Quest Sound Team	panels/ CalendarPanel ChampionshipPanel EffectivePanel HomePanel InfrastructurePanel MailPanel MainPanel MarketPanel MatchPanel MatchSettingsPanel NewUserPanel PlayerPanel QuestsPanel SettingsPanel StadiumPanel SubstitutionDialog TransitionMenu visuals/ HRSButtons HRSColors HRSFonts HRSLabels HRSLanguages GameView

5.2 Architecture de la base de données

BDD Projet PI4 CB2-C



5.3 Outils

- GitLab : système de gestion de versions, CI/CD.
- Java 17 : version assez récente, fiable et portable.
- Maven : gestion du build et des dépendances.
- Swing/AWT : création de l'interface graphique.

5.4 Dépendances

- Checkstyle¹ : vérification de la qualité du code.
- Dotenv² : gestion de variables d'environnement (utile pour la connexion à la base de données).
- Gson³ : lecture et écriture dans un fichier JSON pour une sauvegarde interne de certaines données (id de l'utilisateur, langue, volume et langue) et gestion locale des quêtes.
- JFreeChart⁴ : création du graphiques des statistiques.
- JUnit 5⁵/Mockito⁶ : création et exécution des tests.
- Supabase^{7 8} : stockage des données dans une base de données accessible en ligne.

5.5 Images

- Freepik⁹
- Pngtree¹⁰

5.6 Sons

- Musiques : Pixabay¹¹.
- Effets sonores/bruitages : Mixkit¹².

1. <https://maven.apache.org/plugins>

2. <https://mvnrepository.com/artifact/io.github.cdimascio>

3. <https://mvnrepository.com/artifact/com.google.code.gson/gson>

4. <https://mvnrepository.com/artifact/jfree/jfreechart>

5. <https://mvnrepository.com/artifact/org.junit.jupiter>

6. <https://mvnrepository.com/artifact/org.mockito>

7. <https://mvnrepository.com/artifact/org.postgresql/postgresql>

8. <https://supabase.com>

9. <https://www.freepik.com/app>

10. <https://fr.pngtree.com/>

11. <https://pixabay.com/music>

12. <https://mixkit.co/free-sound-effects>